

5.2.4

Exercise 5.2.4: This grammar generates binary numbers with a “decimal” point:

$$\begin{aligned} S &\rightarrow L . L \mid L \\ L &\rightarrow L B \mid B \\ B &\rightarrow 0 \mid 1 \end{aligned}$$

Design an L-attributed SDD to compute $S.val$, the decimal-number value of an input string. For example, the translation of string 101.101 should be the decimal number 5.625. *Hint:* use an inherited attribute $L.side$ that tells which side of the decimal point a bit is on.

产生式	语义规则
(1) $S \rightarrow L1.L2$	$L1.side = \text{left}$ $L2.side = \text{right}$ $S.val = L1.val + L2.val$
(2) $S \rightarrow L$	$L.side = \text{left}$ $S.val = L.val$
(3) $L \rightarrow L1 B$	$L1.side = L.side$ if ($L.side == \text{left}$) $L.val = L1.val * 2 + B.val$ else $L.val = L1.val + B.val / 2^{(\text{length}(L1))}$
(4) $L \rightarrow B$	if ($L.side == \text{left}$) $L.val = B.val$ else $L.val = B.val / 2$
(5) $B \rightarrow 0$	$B.val = 0$
(6) $B \rightarrow 1$	$B.val = 1$

5.3.1

Exercise 5.3.1: Below is a grammar for expressions involving operator + and integer or floating-point operands. Floating-point numbers are distinguished by having a decimal point.

$$\begin{aligned} E &\rightarrow E + T \mid T \\ T &\rightarrow \text{num} . \text{num} \mid \text{num} \end{aligned}$$

- Give an SDD to determine the type of each term T and expression E .
- Extend your SDD of (a) to translate expressions into postfix notation. Use the unary operator **intToFloat** to turn an integer into an equivalent float.

(a)

产生式	语义规则
(1) $E \rightarrow E1 + T$	if ($E1.type == \text{float}$ or $T.type == \text{float}$) $E.type = \text{float}$ else $E.type = \text{int}$
(2) $E \rightarrow T$	$E.type = T.type$
(3) $T \rightarrow \text{num} . \text{num}$	$T.type = \text{float}$
(4) $T \rightarrow \text{num}$	$T.type = \text{int}$

(b)

产生式	语义规则
(1) $E \rightarrow E1 + T$	if($E1.type == float$ or $T.type == float$) $E.type = float$ else $E.type = int$ if($E1.type == T.type$) $E.val = E1.val + T.val$ else if($E1.type == float$) $E.val = E1.val + intToFloat(T.val)$ else $E.val = intToFloat(E1.val) + T.val$
(2) $E \rightarrow T$	$E.type = T.type$ $E.val = T.val$
(3) $T \rightarrow num.num$	$T.type = float$ $T.val = num.num$
(4) $T \rightarrow num$	$T.type = int$ $T.val = num$

5.4.3

Exercise 5.4.3: The following SDT computes the value of a string of 0's and 1's interpreted as a positive, binary integer.

$$\begin{array}{lcl} B & \rightarrow & B_1 0 \{B.val = 2 \times B_1.val\} \\ & | & B_1 1 \{B.val = 2 \times B_1.val + 1\} \\ & | & 1 \{B.val = 1\} \end{array}$$

Rewrite this SDT so the underlying grammar is not left recursive, and yet the same value of $B.val$ is computed for the entire input string.

$B \rightarrow 1 D \{B.val = 2^D.bit + D.val\}$
 $D \rightarrow 0 D1 \{D.val = D1.val; D.bit = D1.bit + 1\}$
 $D \rightarrow 1 D1 \{D.val = 2^D1.bit + D1.val; D.bit = D1.bit + 1\}$
 $D \rightarrow \epsilon \{D.val = 0; D.bit = 0\}$